

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA5115

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4, BL5)

Debugging & Optimisation Task: 10%

QUESTIONS: 2 Total Marks: 20

Annexure A

Code of Conduct:

I **M Nishanth (192511154)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M Nishanth

Annexure A

Debugging & Optimisation Task

4. **Optimization and Security Analysis of Diffie-Hellman Key Exchange** A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.
 - a Debug incorrect shared key generation due to improper modular arithmetic.
 - b Optimize exponentiation operations for large prime values.
 - c Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. **Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation** A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.
 - a Identify and fix errors in point addition and scalar multiplication logic.
 - b Optimize the implementation using efficient algorithms (e.g., double-and-add).
 - c Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

SIMATS ENGINEERING

ASSESSMENT TOOL 3 – CSA51 CRYPTOGRAPHY AND NETWORK SECURITY (CO2)

Annexure A – Questions 4 & 5: Debugging & Optimisation Solutions

Question 4: Optimization and Security Analysis of Diffie-Hellman Key Exchange

Scenario: A Python implementation of the Diffie-Hellman key exchange is vulnerable to performance bottlenecks and potential security flaws.

A) Debug Incorrect Shared Key Generation (Improper Modular Arithmetic)

Answer: Root cause: the buggy code computes each party's shared key using their OWN public value instead of the OTHER party's public value, so A and B never converge on the same secret.

The most common implementation error in Diffie-Hellman code is swapping which public key is raised to the private exponent. The shared secret must always be computed as (other party's public key)^{7_{ha}_{mq}} (mod p) – not one's own public key. A second frequent error is omitting the modulus argument from pow(), which computes an astronomically large integer before reducing it, wasting time and memory.

X Buggy Code:

```
p, g = 23, 5

a, b = 6, 15          # private keys of A and B

A = pow (g, a, p)      # A's public key
B = pow (g, b, p)      # B's public key

# BUG: each side uses its OWN public key instead of the other party's

shared_key_A = pow (A, a, p)  # should use B, not A
shared_key_B = pow (B, b, p)  # should use A, not B
```

✓ Fixed Code:

```
p, g = 23, 5

a, b = 6, 15          # private keys of A and B

A = pow(g, a, p)       # A's public key
B = pow(g, b, p)       # B's public key

# FIX: each side uses the OTHER party's public key

shared_key_A = pow(B, a, p)  # A combines B's public key with its own private key
shared_key_B = pow(A, b, p)  # B combines A's public key with its own private key
```

Verified Output (executed):

```
BUGGY -> shared_A: 13 shared_B: 20 match: False
FIXED -> shared_A: 2 shared_B: 2 match: True
```

Running both versions confirms the diagnosis: the buggy version produces two different “shared” keys (13 and 20), which would make all subsequent encrypted communication fail; the fixed version produces the identical key (2) on both sides, exactly matching the manually verified result from Q4 of Assessment Tool 1.

B) Optimize Exponentiation for Large Prime Values

Answer: Use Python's built-in three-argument `pow (base, exp, mod)`, which performs modular reduction at every squaring step (square-and-multiply) instead of computing the full power first.

A naive/unoptimized implementation computes `base ** exp` as a full integer before applying `% mod`. For large primes (as used in real Diffie-Hellman, typically 2048 bits), this intermediate number can have millions of digits, making the operation extremely slow and memory-heavy. The correct optimization is to reduce modulo `p` at every step of a square-and-multiply loop – which is exactly what Python's built-in `pow(base, exp, mod)` already does internally in optimized C code.

Optimized modular exponentiation (manual square-and-multiply, for reference):

```
def fast_modexp(base, exp, mod):  
  
    """O(log exp) modular exponentiation – reduces mod at every step."""  
  
    result = 1  
  
    base = base % mod  
  
    while exp > 0:  
  
        if exp & 1:          # if current bit is 1  
  
            result = (result * base) % mod  
  
        exp >>= 1           # shift to next bit  
  
        base = (base * base) % mod    # square and reduce  
  
    return result  
  
# In production code, prefer the built-in (implemented in optimized C):  
  
shared_key = pow(B, a, p)
```

Benchmark – unreduced (naive) power vs. built-in pow(), exponent = 200,000:

```
unreduced naive pow:  0.015999 sec  
  
built-in pow():      0.000038 sec  
  
results match: True  
  
-> built-in pow () is ~420x faster for this exponent size
```

This benchmark (executed directly) confirms the expected complexity improvement: computing the full power before reducing scales with the size of the final (huge) intermediate number, while modular reduction at every step keeps all intermediate values bounded by the modulus, giving $O(\log \text{exp})$ multiplications of fixed-size numbers instead of one enormous multiplication.

C) Mitigating Man-in-the-Middle Attacks and Performance Impact

Answer: Add authentication of the exchanged public values (e.g., an HMAC or digital signature tied to a pre-shared/certified key); the added overhead is negligible compared to the exponentiation cost.

Plain Diffie-Hellman has no way to verify that a received public value genuinely came from the claimed party, which is what allows a Man-in-the-Middle (MITM) attacker to substitute their own public values (see Q4 of Assessment Tool 1 for the full attack walkthrough). The fix is to authenticate each public value before it is trusted, for example by attaching a keyed-hash (HMAC) computed with a secret the two parties already share via a trusted channel, or a digital signature verifiable against a certificate issued by a trusted Certificate Authority (as in the Station-to-Station protocol).

Extended code – authenticated public-key exchange using HMAC:

```
import hmac, hashlib, time

# A pre-shared authentication key, established once via a trusted channel
# (in production this role is normally played by a certificate/PKI signature)
auth_key = b'shared-trust-secret'

def sign_public_value(pub_value, auth_key):
    """Attach an HMAC tag so the receiver can verify authenticity."""
    msg = str(pub_value).encode()
    tag = hmac.new(auth_key, msg, hashlib.sha256).hexdigest()
    return pub_value, tag

def verify_public_value(pub_value, tag, auth_key):
    """Reject the exchange if the tag does not match (tampering detected)."""
    expected = hmac.new(auth_key, str(pub_value).encode(), hashlib.sha256).hexdigest()
```

```

return hmac.compare_digest(expected, tag)

# --- A sends its public key, authenticated ---

A_pub, A_tag = sign_public_value(A, auth_key)

# --- B receives and verifies before trusting it ---

if verify_public_value(A_pub, A_tag, auth_key):

    shared_key_B = pow(A_pub, b, p) # only proceed if authentic
else:

    raise ValueError('Public key failed authentication - possible MITM attempt')

```

Performance impact evaluation: computing and verifying an HMAC-SHA256 tag takes on the order of microseconds – several orders of magnitude cheaper than the modular exponentiation itself (which, per the benchmark above, already completes in well under a millisecond even for large exponents). Adding authentication therefore closes the MITM vulnerability at effectively negligible additional cost, making it a strongly favourable trade-off. A full certificate-based signature (e.g., RSA or ECDSA) would add somewhat more overhead than a simple HMAC but remains small relative to the security benefit, and is the approach used in real-world protocols such as TLS.

Question 5: Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation

Scenario: A Python script implementing Elliptic Curve Cryptography for key exchange produces incorrect results for certain curve parameters.

A) Identify and Fix Errors in Point Addition and Scalar Multiplication Logic

Answer: Root cause: the point-doubling/addition formula used ordinary (real-number) division instead of modular inverse, and/or failed to special-case point doubling and the point at infinity.

On an elliptic curve over a finite field, the slope in the point addition/doubling formula must be computed using the modular inverse (since ordinary division is undefined in modular arithmetic), and doubling ($P + P$) must be handled with a different formula than adding two distinct points ($P + Q$, $P \neq Q$). A further common bug is not treating the identity element (“point at infinity”, represented as `None`) as a special case.

✗ Buggy Code (point doubling):

```
def point_double_buggy(P, a, p):  
    x1, y1 = P  
    m = (3 * x1**2 + a) / (2 * y1)    # BUG: real division, not modular inverse  
    x3 = (m**2 - 2*x1) % p  
    y3 = (m * (x1 - x3) - y1) % p  
    return (x3, y3)
```

✓ Fixed Code (full, correctly handling addition, doubling, and infinity):

```
def mod_inverse(k, p):  
    """Modular inverse via Fermat's Little Theorem (p must be prime)."""  
    return pow(k, p - 2, p)  
  
def point_add(P, Q, a, p):  
    if P is None:                # identity element (point at infinity)  
        return Q  
    if Q is None:  
        return P  
    x1, y1 = P  
    x2, y2 = Q  
  
    if x1 == x2 and (y1 + y2) % p == 0:
```

```

return None                # P + (-P) = point at infinity

if P == Q:                  # point doubling case
    m = (3 * x1**2 + a) * mod_inverse((2 * y1) % p, p) % p
else:                       # point addition case
    m = (y2 - y1) * mod_inverse((x2 - x1) % p, p) % p

x3 = (m**2 - x1 - x2) % p
y3 = (m * (x1 - x3) - y1) % p

return (x3, y3)

```

Verified Output (executed on curve $y^2 = x^3 + 2x + 2 \bmod 17$, point $P = (5,1)$):

BUGGY 2P : (10.25, 0.875) <-- not a valid point on the curve (non-integer!)

FIXED 2P : (6, 3)

Verification : $y^2 \bmod p = 9$ | $x^3 + ax + b \bmod p = 9$ valid: True

The buggy version's use of true division produces a non-integer point (10.25, 0.875), which is meaningless on a discrete curve over a finite field – confirming the bug. The fixed version produces (6, 3), which is verified to satisfy the curve equation exactly, confirming the correction is mathematically valid.

B) Optimize Using Double-and-Add

Answer: Replace $O(k)$ repeated point addition with $O(\log k)$ double-and-add scalar multiplication.

A naive implementation of $k \cdot P$ adds the point P to itself k times in a loop – requiring k point-addition operations. The standard optimization, double-and-add (directly analogous to square-and-multiply for modular exponentiation), processes the binary representation of k , doubling the running point at every bit and adding it in only when that bit is 1, reducing the operation count from $O(k)$ to $O(\log_2 k)$.

Optimized scalar multiplication:

```
def scalar_mult_naive(k, P, a, p):  
  
    """O(k) – unoptimized repeated addition."""  
  
    result = None  
  
    for _ in range(k):  
  
        result = point_add(result, P, a, p)  
  
    return result  
  
def scalar_mult_fast(k, P, a, p):  
  
    """O(log k) – double-and-add."""  
  
    result = None  
  
    addend = P  
  
    while k:  
  
        if k & 1:  
  
            result = point_add(result, addend, a, p)  
  
            addend = point_add(addend, addend, a, p) # double  
  
        k >>= 1 # next bit  
  
    return result
```

Benchmark – executed on a curve with $p \approx 10^6$, scalar $k = 5000$:

naive O(k) repeated addition: 0.009257 sec

double-and-add O(log k): 0.000040 sec

results match: True

-> double-and-add is ~230x faster for $k = 5000$, and the gap

widens further as k grows (real ECC scalars are ~256 bits)

C) ECC vs RSA: Efficiency, Security Strength, and Suitability for Resource-Constrained Environments

Answer: ECC is more computationally efficient than RSA at equivalent security, and is the better choice for resource-constrained (e.g., IoT) environments.

Aspect	RSA	ECC
Underlying hard problem	Integer factorization	Elliptic curve discrete logarithm
Key size for ≈ 128 -bit security	3072 bits	256 bits
Core operation cost	Large modular exponentiation – grows $\sim$ cubically with key length	Scalar multiplication via double-and-add – $O(\log k)$ point operations on much smaller numbers
Memory / bandwidth	Larger keys and ciphertexts to store and transmit	Much smaller keys and signatures for the same security
Suitability for constrained devices	Higher CPU, memory, and battery cost	Well suited – lower CPU, memory, and power usage

Justification: for a resource-constrained environment (limited CPU, memory, and battery, as in IoT devices), ECC provides equivalent cryptographic strength to RSA while using a dramatically smaller key size (e.g., 256-bit ECC $\approx$ 3072-bit RSA in security strength). Combined with the double-and-add optimization demonstrated above – which brings scalar multiplication down to $O(\log k)$ – ECC operations complete faster and consume less memory and power than the large modular exponentiations RSA requires. This makes ECC the clearly preferable cryptosystem for embedded and IoT key-exchange scenarios, which is also why it is the industry-standard choice (e.g., ECDH, ECDSA) for such environments today.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrate s strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrate d
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lack's structure and proper documentation	Poorly written code with no documentation